

FilePond properties

The FilePond instance exposes a set of properties that enable us to update its state.



Combine FilePond with Pintura and create the perfect image edit and upload experience.

[Learn more about Pintura](#)

Core properties [↗](#)

The FilePond core module exposes the following properties.

PROPERTY	DEFAULT VALUE	DESCRIPTION
element	<code>null</code>	The root element of the FilePond instance. This property has no setter
status	<code>0</code>	Returns the current status of the FilePond instance, use the <code>FilePond.Status</code> enum to determine the status. Has no setter.
name	<code>'filepond'</code>	The input field name to use
className	<code>null</code>	Additional CSS class to add to the root element
required	<code>false</code>	Sets the required attribute to the output field
disabled	<code>false</code>	Sets the disabled attribute to the output field
captureMethod	<code>null</code>	Sets the given value to the <code>capture</code> attribute
<u>files</u>	A list of file locations that should be loaded Immediately	
allowDrop	<code>true</code>	Enable or disable drag n' drop
allowBrowse	<code>true</code>	Enable or disable file browser
allowPaste	<code>true</code>	Enable or disable pasting of files. Pasting files is not supported on all browsers.
allowMultiple	<code>false</code>	Enable or disable adding multiple files
allowReplace	<code>true</code>	Allow drop to replace a file, only works when <code>allowMultiple</code> is <code>false</code>
allowRevert	<code>true</code>	Enable or disable the revert processing button
allowRemove	<code>true</code>	When set to false the remove button is hidden and disabled
allowProcess	<code>true</code>	Enable or disable the process button

PROPERTY	DEFAULT VALUE	DESCRIPTION
<code>allowReorder</code>	<code>false</code>	Allow users to reorder files with drag and drop interaction. Note that this only works in single column mode. It also only works on browsers that support pointer events.
<code>storeAsFile</code>	<code>false</code>	Tells FilePond to store files in hidden file input elements so they can be posted along with normal form post. This only works if the browser <u>supports the <code>DataTransfer</code> constructor</u> , this is the case on Firefox, Chrome, Chromium powered browsers and Safari version 14.1 and higher.
<code>forceRevert</code>	<code>false</code>	Set to <code>true</code> to require the file to be successfully reverted before continuing
<code>maxFiles</code>	<code>null</code>	The maximum number of files that the pond can handle
<code>maxParallelUploads</code>	<code>2</code>	The maximum number of files that can be uploaded in parallel
<code>checkValidity</code>	<code>false</code>	Set to <code>true</code> to enable custom validity messages. FilePond will throw an error when a parent form is submitted and it contains invalid files.
<code>itemInsertLocation</code>	<code>'before'</code>	Set to <code>'after'</code> to add files to end of list (when dropped at the top of the list or added using browse or paste), set to <code>'before'</code> to add files at start of list. Set to a compare function to automatically <u>sort items</u> when added
<code>itemInsertInterval</code>	<code>75</code>	The interval to use before showing each item being added to the list
<code>fileSizeBase</code>	<code>1000</code>	The base to use for file size calculations, this is only used for displaying file sizes.
<code>credits</code>	<code>[url, label]</code>	Shows <u>Powered by PQINA</u> in footer

Drag n' Drop

PROPERTY	DEFAULT VALUE	DESCRIPTION
<code>dropOnPage</code>	<code>false</code>	FilePond will catch all files dropped on the webpage
<code>dropOnElement</code>	<code>true</code>	Require drop on the FilePond element itself to catch the file.
<code>dropValidation</code>	<code>false</code>	When enabled, files are validated before they are dropped. A file is not added when it's invalid.
<code>ignoredFiles</code>	<code>['.ds_store', 'thumbs.db', 'desktop.ini']</code>	Ignored file names when handling dropped directories. Dropping directories is not supported on all browsers.

Server

PROPERTY	DEFAULT VALUE	DESCRIPTION
server	<code>null</code>	A <u>server configuration object</u> describing how FilePond should interact with the server.
instantUpload	<code>true</code>	Immediately upload new files to the server
chunkUploads	<code>false</code>	Enable chunked uploads, when enabled will automatically cut up files in <code>chunkSize</code> chunks before upload
chunkForce	<code>false</code>	Force chunks even for files smaller than the set <code>chunkSize</code>
chunkSize	<code>5000000</code>	The size of a chunk in bytes
chunkRetryDelays	<code>[500, 1000, 3000]</code>	Amount of times, and delays, between retried uploading of a chunk

Labels [↗](#)

PROPERTY	DEFAULT VALUE	DESCRIPTION
labelDecimalSeparator	<code>undefined</code>	The decimal separator used to render numbers. By default this is determined automatically.
labelThousandsSeparator	<code>undefined</code>	The thousands separator used to render numbers. By default this is determined automatically.
labelIdle	<code>'Drag & Drop your files or Browse '</code>	Default label shown to indicate this is a drop area. FilePond will automatically bind browse file events to the element with CSS class <code>.filepond--label-action</code>
labelInvalidField	<code>'Field contains invalid files'</code>	Label shown when the field contains invalid files and is validated by the parent form.
labelFileWaitingForSize	<code>'Waiting for size'</code>	Label used while waiting for file size information
labelFileSizeNotAvailable	<code>'Size not available'</code>	Label used when no file size information was received
labelFileLoading	<code>'Loading'</code>	Label used while loading a file
labelFileLoadError	<code>'Error during load'</code>	Label used when file load failed
labelFileProcessing	<code>'Uploading'</code>	Label used when uploading a file
labelFileProcessingComplete	<code>'Upload complete'</code>	Label used when file upload has completed
labelFileProcessingAborted	<code>'Upload cancelled'</code>	Label used when upload was cancelled
labelFileProcessingError	<code>'Error during upload'</code>	Label used when something went wrong during file upload

PROPERTY	DEFAULT VALUE	DESCRIPTION
labelFileProcessingRevertError	'Error during revert'	Label used when something went wrong during reverting the file upload
labelFileRemoveError	'Error during remove'	Label used to indicate something went wrong when removing the file
labelTapToCancel	'tap to cancel'	Label used to indicate to the user that an action can be cancelled.
labelTapToRetry	'tap to retry'	Label used to indicate to the user that an action can be retried.
labelTapToUndo	'tap to undo'	Label used to indicate to the user that an action can be undone.
labelButtonRemoveItem	'Remove'	Label used for remove button
labelButtonAbortItemLoad	'Abort'	Label used for abort load button
labelButtonRetryItemLoad	'Retry'	Label used for retry load button
labelButtonAbortItemProcessing	'Cancel'	Label used for abort upload button
labelButtonUndoItemProcessing	'Undo'	Label used for undo upload button
labelButtonRetryItemProcessing	'Retry'	Label used for retry upload button
labelButtonProcessItem	'Upload'	Label used for upload button

SVG Icons [↗](#)

PROPERTY	DEFAULT VALUE	DESCRIPTION
iconRemove	'<svg>...</svg>'	The icon used for remove actions
iconProcess	'<svg>...</svg>'	The icon used for process actions
iconRetry	'<svg>...</svg>'	The icon used for retry actions
iconUndo	'<svg>...</svg>'	The icon used for undo actions

Callbacks [↗](#)

PROPERTY	FUNCTION	DESCRIPTION
oninit()	FilePond instance has been created and is ready.	
onwarning(error[, file, status])	FilePond instance throws a warning. For instance when the maximum amount of files has been reached. Optionally receives file if error is related to a file object.	
onerror(error[, file, status])	FilePond instance throws an error. Optionally receives file if error is related to a file object.	
oninitfile(file)	Created file item	

PROPERTY	FUNCTION	DESCRIPTION
<code>onaddfilestart(file)</code>	Started file load	
<code>onaddfileprogress(file, progress)</code>	Made progress loading a file	
<code>onaddfile(error, file)</code>	If no error, file has been succesfully loaded	
<code>onprocessfilestart(file)</code>	Started processing a file	
<code>onprocessfileprogress(file, progress)</code>	Made progress processing a file	
<code>onprocessfileabort(file)</code>	Aborted processing of a file	
<code>onprocessfilerevert(file)</code>	Processing of a file has been reverted	
<code>onprocessfile(error, file)</code>	If no error, Processing of a file has been completed	
<code>onprocessfiles()</code>	Called when all files in the list have been processed	
<code>onremovefile(error, file)</code>	File has been removed.	
<code>onpreparefile(file, output)</code>	File has been transformed by the transform plugin or another plugin subscribing to the prepare_output filter. It receives the file item and the output data.	
<code>onupdatefiles(files)</code>	A file has been added or removed, receives a list of file items	
<code>onactivatefile(file)</code>	Called when a file is clicked or tapped	
<code>onreorderfiles(files, origin, target)</code>	Called when the files list has been reordered, receives current list of files (reordered) plus file origin and target index.	

Hooks

PROPERTY	DESCRIPTION
<code>beforeDropFile(file)</code>	FilePond is about to allow this item to be dropped, it can be a URL or a File object. Return true or false depending on if you want to allow the item to be dropped.
<code>beforeAddFile(item)</code>	FilePond is about to add this file, return false to prevent adding it, or return a Promise and resolve with true or false.
<code>beforeRemoveFile(item)</code>	FilePond is about to remove this file, return false to prevent removal, or return a Promise and resolve with true or false.

Styles

PROPERTY	DEFAULT VALUE	DESCRIPTION
<code>stylePanelLayout</code>	<code>null</code>	Set a different layout render mode. Can be either 'integrated', 'compact', and/or 'circle'. Compact mode will remove padding, integrated mode is used to render FilePond as part of a bigger element. Circle mode adjusts the item position offsets so buttons and progress indicators don't fall outside of the circular shape.

PROPERTY	DEFAULT VALUE	DESCRIPTION
<code>stylePanelAspectRatio</code>	<code>null</code>	Set a forced aspect ratio for the FilePond drop area. Useful to make the drop area take up a fixed amount of space. For example when cropping a single square image. Accepts human readable aspect ratios like <code>'1:1'</code> or numeric aspect ratios like <code>0.75</code> .
<code>styleButtonRemoveItemPosition</code>	<code>'left'</code>	The position of the remove item button, <code>'left'</code> , <code>'center'</code> , <code>'right'</code> , and/or <code>'bottom'</code> .
<code>styleButtonProcessItemPosition</code>	<code>'right'</code>	The position of the process item button, <code>'left'</code> , <code>'center'</code> , <code>'right'</code> , and/or <code>'bottom'</code> .
<code>styleLoadIndicatorPosition</code>	<code>'right'</code>	The position of the load indicator, <code>'left'</code> , <code>'center'</code> , <code>'right'</code> , and/or <code>'bottom'</code> .
<code>styleProgressIndicatorPosition</code>	<code>'right'</code>	The position of the progress indicator, <code>'left'</code> , <code>'center'</code> , <code>'right'</code> , and/or <code>'bottom'</code> .
<code>styleItemPanelAspectRatio</code>	<code>null</code>	Set a forced aspect ratio for the file items. Useful when rendering cropped or fixed aspect ratio images in grid view, this will improve performance as FilePond will know beforehand the size of the item to render.

Files

You can populate FilePond with a set of initial files using the `files` property. This property is suitable for restoring earlier temporary uploads or already uploaded files. While the `files` property can also be populated with new files, it is advised to add new files using only the `addFile` and `addFiles` methods.

The `files` property takes an array of file references, it accepts the same formats as the `addFile` method.

- Local and Remote URLs
- DataURLs
- Blobs or File objects

It extends this selection with the option to add server files.

Adding a temporary server file. The `limbo` type will direct the load request to the [server API restore end point](#).

```

1  const pond = FilePond.create({
2    files: [
3      {
4        // the server file reference
5        source: '12345',
6
7        // set type to limbo to tell FilePond this is a temp file
8        options: {
9          type: 'limbo',
10       },
11     ],

```

```

12     },
13   ],
14 });

```

Adding an already uploaded server file. The `local` type will direct the load request to the server API load end point.

```

1  const pond = FilePond.create({
2    files: [
3      {
4        // the server file reference
5        source: '12345',
6
7        // set type to local to indicate an already uploaded file
8        options: {
9          type: 'local',
10       },
11     },
12   ],
13 });

```

You can also mock the file by supplying file information in the `options` object, this will be the same information that FilePond otherwise fetches from the server. FilePond will now create a mock file item, and won't load the file data.

```

1  const pond = FilePond.create({
2    files: [
3      {
4        // the server file reference
5        source: '12345',
6
7        // set type to local to indicate an already uploaded file
8        options: {
9          type: 'local',
10
11          // mock file information
12          file: {
13            name: 'my-file.png',
14            size: 3001025,
15            type: 'image/png',
16          },
17        },
18      },
19    ],
20  });

```

Note that this will impact the way plugins render or process the file as no actual file data is available.

If you want to supply initial metadata to a loaded file (accessible using `setMetadata` and `getMetadata` on the file item) you can do so with the `metadata` property. The following example will create a `date` entry in the FilePond file item metadata object.

```

1  const pond = FilePond.create({
2    files: [
3      {
4        // the server file reference
5        source: '12345',
6
7        // set type to local to indicate an already uploaded file
8        options: {
9          type: 'local',
10
11          // file initial metadata
12          metadata: {
13            date: '2018-10-5T12:00',
14          },
15        },
16      },
17    ],
18  });
19
20  // get the data of the first file
21  const date = pond.getFile().getMetadata('date');

```

Fitting with the progressive enhancement strategy FilePond adheres to, it's also possible to feed FilePond an initial file using HTML.

The example below might seem elaborate at first, maybe it is a bit. But should JavaScript fail to load, for whatever reason, the user will still be able to remove files (by unchecking them) and add new files. The resulting form object when posted will contain all information required to update any server state.

```

1  <fieldset>
2    <legend>Files</legend>
3
4    <!-- a list of already uploaded files -->
5    <ul>
6      <li>
7        <label>
8          <input value="foo.jpg" data-type="local" checked type="checkbox" />
9          foo.jpeg
10         </label>
11       </li>
12      <li>
13        <label>
14          <input value="bar.png" data-type="local" checked type="checkbox" />
15          bar.png
16        </label>
17      </li>
18    </ul>
19
20    <!-- our filepond input -->
21    <input type="file" name="filepond" required multiple />
22  </fieldset>
23
24  <script>
25    // get a reference to the fieldset element
26    const fieldsetElement = document.querySelector('fieldset');
27
28    // create a FilePond instance at the fieldset element location
29    const pond = FilePond.create(fieldsetElement);
30

```

```

31 |
32 |     // files have been gathered
33 |     console.log(pond.files); // [{ source: 'foo.jpeg' }, { source: 'bar.png' }]
34 | </script>

```

And this is how that HTML renders without styles. It's not super fancy, but it allows the user to upload or change his file selection, even without JavaScript.

```

1 | <fieldset>
2 |   <legend>Files</legend>
3 |
4 |   <!-- a list of already uploaded files -->
5 |   <ul>
6 |     <li>
7 |       <label>
8 |         <input value="foo.jpg" data-type="local" checked type="checkbox" />
9 |         foo.jpeg
10 |       </label>
11 |     </li>
12 |     <li>
13 |       <label>
14 |         <input value="bar.png" data-type="local" checked type="checkbox" />
15 |         bar.png
16 |       </label>
17 |     </li>
18 |   </ul>
19 |
20 |   <!-- our filepond input -->
21 |   <input type="file" required multiple />
22 | </fieldset>

```

Disabling Credits

FilePond is given to you for free. Please support the project by leaving the tiny *Powered by PQ/INA* footer in place, it's a nofollow link.

If you intend to hide the *Powered by PQ/INA* footer, please support the project by linking to <https://pqina.nl> somewhere else on your website.

You can also restyle the footer by adjusting the `.filepond--credits` CSS class.

Your support helps keep the project alive.

Of course you can still disable the credits message by setting the `credits` property to `false`. If you do decide to remove the credits message and use FilePond in a commercial project please consider [donating](#) to keep the project alive.